EPFL Department of Mathematics

Course Project: MPI Parallelization of the Conjugate Gradient Algorithm

MATH-454: Parallel and High-Performance Computing

Author: Selim SHERIF

April 16, 2025

Brief Introduction

This project addresses the parallelization of the Conjugate Gradient (CG) method for solving large, sparse linear systems. The work begins with profiling a sequential implementation to identify performance-critical sections. Based on this analysis, the sequential fraction is estimated, and theoretical predictions of speedup and efficiency are made using Amdahl's and Gustafson's laws. A parallel version of the algorithm is then developed using MPI, followed by strong and weak scaling experiments to evaluate performance. Finally, the measured results are compared with the theoretical predictions and analyzed accordingly.

Profiling

Before attempting parallelization and performance optimization, it is essential to identify the computational bottlenecks in the code. To this end, profiling was conducted using `perf`, a command-line tool integrated into the Linux environment. The profiling results are summarized below.

- The majority of computational time is spent in the `.solve` method, with matrix-vector multiplication (`Mat_vec`) alone accounting for approximately **67%** of the recorded samples.
- Significant overhead is attributed to `daxpy_k_PRESCOTT` (BLAS-related, **24.2%**) and `std::vec` allocations (**8%**), reflecting the cost of repeated vector operations and dynamic memory management.

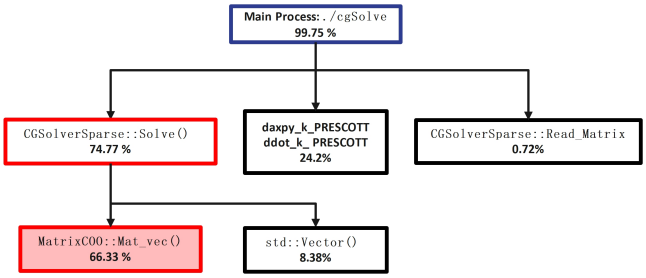

Figure 1: Perf-based profiling output. Full report available in the annex.

- Matrix reading contributes less than **0.5%** of total execution time and is therefore considered negligible in this performance analysis.

With this in mind, it is clear that the focus will be on parallelizing the `.solve` method, with particular emphasis on optimizing the `Mat_Vec` method.

Code Architecture

As mentioned above, the only significant section identified for parallelization based on the profiling analysis is the matrix-vector multiplication. Therefore, the only part of the code that has been parallelized is the `mat_vec` function using `MPI_Allreduce` with a `SUM` operation at the end to combine partial results from all nodes. Each node receives an equal number of non-zero elements, and the result of the matrix-vector product is obtained by summing the results of each "local" product computed independently.

Results

Strong Scaling

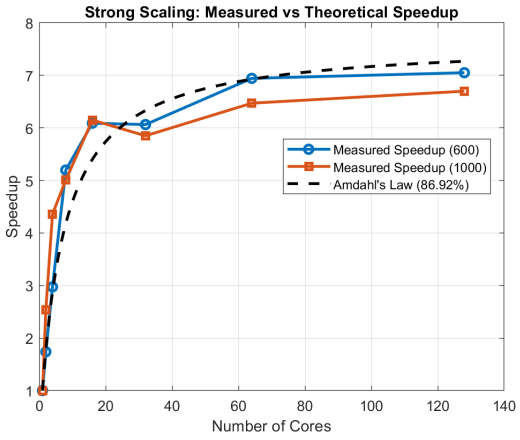

Figure 2: Measured speedup under strong scaling with theoretical prediction using Amdahl's Law.

The parallel fraction $P = 0.8692$ was determined by profiling the total execution time spent in the `mat_vec` function during a fully serial run using the `lap2D_5pt_n1000.mtx` matrix. The same test was also performed on the `lap2D_5pt_n600.mtx` matrix for comparison.

Note that the compiler optimization flag `-O2` was deliberately omitted throughout all tests. This is because the compiler's optimizations interfered with the parallelization logic—in some cases, it optimized away computation so aggressively that it masked the actual parallel workload.

The measured speedup curve exhibits the classic behavior described by Amdahl's Law. Speedup increases initially with additional cores, but flattens as the sequential portion of the computation begins to dominate. The theoretical upper bound is given by:

$$S_{\text{Amdahl}}(N) = \frac{1}{(1-P) + \frac{P}{N}}, \quad P = 0.8692$$

In both matrix cases, the measured speedup closely follows Amdahl's Law, with the `n1000` case converging to a slightly lower speedup due to its larger computational load and the increased impact of non-parallelizable components.

Weak Scaling Efficiency

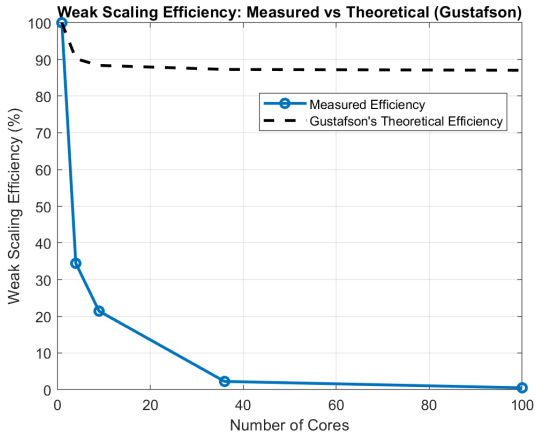

Figure 3: Measured efficiency under weak scaling. Ideally, efficiency should remain close to 100% as work per core stays constant.

This test was conducted using the `n1000` matrix, with the number of cores increased proportionally to the number of non-zero elements per node. The same parallel fraction $P = 0.8692$ was used throughout.
The weak scaling efficiency drops sharply with increasing core count, deviating from Gustafson's Law,

$$S_{\text{Gustafson}}(N) = N - (1-P)(N-1),$$

which assumes a constant sequential portion as the problem size grows. Ideally, runtime remains constant and efficiency,

$$E_{\text{weak}}(N) = \frac{T(1)}{T(N)},$$

stays near 1. Instead, runtime increases significantly, revealing poor scalability.
A likely bottleneck is the `MPI_Allreduce` used in `mat_vec`, which scales poorly with both message size and process count. Additional factors include memory contention, synchronization overhead, and the sparse matrix band layout of the Laplace problem. Furthermore, the matrix's tridiagonal sparsity limits the available parallel work, resulting in a low communication-to-computation ratio. Although the algorithm is parallelizable in theory, the current implementation fails to scale efficiently under weak scaling at larger core counts.

Concluding remarks

The experiment demonstrated both the strengths and limitations of MPI-based parallelization for the Conjugate Gradient algorithm, highlighting the central role of the `mat_vec` operation. The strong scaling results closely follow Amdahl's Law, while the weak scaling tests reveal significant communication bottlenecks and poor scalability at higher core counts.

Future improvements could focus on the following directions:

- **Parallelizing additional components** of the solver pipeline, such as vector updates and dot products, to reduce serial overhead.
- **Employing ghost cells or domain decomposition** strategies to localize communication and minimize the reliance on costly collective operations like `MPI_Allreduce`.
- **Exploring hybrid parallelism** (MPI combined with OpenMP or multithreading) to better exploit shared-memory architectures and reduce inter-process communication.